CPY Compiler — Project Based Learning Documentation

A custom compiled language (`.cpy`) that goes through Lexing → Parsing → Semantic Analysis → Bytecode Compilation → VM Execution.

Project Division

Part	Student	Component
1	Aditya	Virtual Machine + Bytecode I/O
2	Megha	Semantic Analyzer + Bytecode Compiler
3	Sameer	Parser (AST Construction)
4	Vridhi	Lexer (Tokenization)

Part 1 — Aditya : Virtual Machine + Bytecode I/O

Files: `vm/VM.java`, `compiler/BytecodeWriter.java`, `compiler/BytecodeReader.java`, `Main.java`

The VM executes the compiled bytecode. BytecodeWriter serializes instructions to a `.cpyc` file. BytecodeReader deserializes them back. Main.java wires the entire pipeline together.

VM.java — Architecture

```
private final List<Instruction> program;
private final Stack<Object> stack = new Stack<>();
private final Map<String, Object> env = new HashMap<>();
private int pc = 0;
```

The VM has three core components: - `stack` — the operand stack where intermediate values live during computation - `env` — the environment (variable store) mapping names to their current values - `pc` — the program counter, an index into the instruction list that tracks which instruction executes next

---

VM.java — Main Execution Loop

```
public void run() {
    while (pc < program.size()) {
```

```
        Instruction instr = program.get(pc);
        execute(instr);
        if (instr.opCode == OpCode.HALT) break;
    }
}
```

The loop fetches the instruction at `pc`, executes it (which usually increments `pc` internally), and stops when `HALT` is reached. Jump instructions modify `pc` directly instead of incrementing it, which is how control flow works.

VM.java — Arithmetic with Type Checking

```
case ADD: {
    Object b = stack.pop();
    Object a = stack.pop();
    if (a instanceof Double && b instanceof Double) {
        stack.push((double) a + (double) b);
    } else if (a instanceof String || b instanceof String) {
        stack.push(stringify(a) + stringify(b));
    } else {
        throw error("ADD requires two numbers or at least one string");
    }
    pc++;
    break;
}
```

`ADD` handles two cases: numeric addition and string concatenation. If either operand is a String, both are stringified and concatenated. This allows `"Hello " + name` to work naturally. All other arithmetic ops (`SUB`, `MUL`, `DIV`) require both operands to be `Double` and call `checkNumbers()` to enforce this.

VM.java — Control Flow Instructions

```
case JUMP:
    pc = Integer.parseInt(instr.operand);
    break;

case JUMP_IF_FALSE: {
    Object condition = stack.pop();
    if (!isTruthy(condition)) {
        pc = Integer.parseInt(instr.operand);
    } else {
        pc++;
    }
```

```
    break;
}
```

`JUMP` unconditionally sets `pc` to the target index. `JUMP_IF_FALSE` pops the top of the stack and checks its truthiness — if falsy, it jumps; otherwise it falls through to the next instruction. The target index was patched in by the compiler's `patchJump()` method.

VM.java — Array Operations

```
case MAKE_ARRAY: {
    int count = Integer.parseInt(instr.operand);
    Object[] temp = new Object[count];
    for (int i = count - 1; i >= 0; i--) {
        temp[i] = stack.pop();
    }
    List<Object> array = new ArrayList<>();
    for (Object o : temp) array.add(o);
    stack.push(array);
    pc++;
    break;
}

case ARRAY_LOAD: {
    Object idxVal = stack.pop();
    Object arrVal = stack.pop();
    List<Object> list = (List<Object>) arrVal;
    stack.push(list.get(toIndex(idxVal)));
    pc++;
    break;
}
```

`MAKE_ARRAY` pops `count` elements from the stack. Since elements were pushed left-to-right, the last element is on top — the `temp` array reverses this to restore the original order. `ARRAY_LOAD` pops the index and the array, then pushes the element at that index.

BytecodeWriter.java — Serialization

```
public static void write(List<Instruction> instructions, String filename) throws IOExceptio
    try (BufferedWriter writer = new BufferedWriter(new FileWriter(filename))) {
        writer.write("#CPY_BYTECODE v1.0");
        writer.newLine();
        for (Instruction instr : instructions) {
```

```
            if (instr.operand == null) {
                writer.write(instr.opCode.name());
            } else if (instr.opCode == OpCode.CONST_STR) {
                writer.write(instr.opCode.name() + " \"" + escapeString(instr.operand) + "\"
            } else {
                writer.write(instr.opCode.name() + " " + instr.operand);
            }
            writer.newLine();
        }
    }
}
```

The `.cpyc` format is human-readable text: one instruction per line, with the opcode name followed by an optional operand. String operands are wrapped in quotes and escaped (handling `\n`, `\t`, `\"`, etc.) so they survive the round-trip through the file. The header `#CPY_BYTECODE v1.0` lets the reader validate the file format.

Main.java — Full Pipeline

```
private static void compile(String sourceFile) {
    String source = readFile(sourceFile);
    List<Token> tokens    = new Lexer(source).scanTokens();       // 1. Lex
    List<Stmt>  stmts     = new Parser(tokens).parse();           // 2. Parse
    new SemanticAnalyzer().analyze(stmts);                        // 3. Semantic check
    List<Instruction> bc  = new BytecodeCompiler().compile(stmts);// 4. Compile
    String outFile = sourceFile.replaceAll("\\.cpy$", ".cpyc");
    BytecodeWriter.write(bc, outFile);                            // 5. Write .cpyc
}

private static void run(String bytecodeFile) {
    List<Instruction> bytecode = BytecodeReader.read(bytecodeFile);
    new VM(bytecode).run();
}
```

`Main` exposes two commands: `compile` and `run`. The compile pipeline chains all five stages. The run pipeline reads the `.cpyc` file and hands it directly to the VM. This separation means you can compile once and run many times without re-parsing.

Git Commands — Aditya

```
git add Compiler/src/vm/VM.java
git add Compiler/src/compiler/BytecodeWriter.java
```

```
git add Compiler/src/compiler/BytecodeReader.java
git add Compiler/src/Main.java
```

Part 2 — Megha : Semantic Analyzer + Bytecode Compiler

Files: `semantic/SemanticAnalyzer.java`, `compiler/BytecodeCompiler.java`, `compiler/OpCode.java`, `compiler/Instruction.java`

The Semantic Analyzer validates the AST for logical correctness. The Bytecode Compiler then translates the validated AST into a linear sequence of stack-machine instructions.

SemanticAnalyzer.java — Symbol Table

```
private final Map<String, String> symbolTable = new HashMap<>();
private final List<String> errors = new ArrayList<>();

public void analyze(List<Stmt> statements) {
    for (Stmt stmt : statements) {
        analyzeStmt(stmt);
    }
    if (!errors.isEmpty()) {
        StringBuilder sb = new StringBuilder("Semantic errors:\n");
        for (String err : errors) sb.append("  • ").append(err).append("\n");
        throw new RuntimeException(sb.toString());
    }
}
```

The `symbolTable` maps variable names to their type (currently `"any"` since CPY is dynamically typed). Errors are collected into a list rather than thrown immediately — this allows all semantic errors to be reported at once instead of stopping at the first one.

SemanticAnalyzer.java — Variable Declaration Check

```
} else if (stmt instanceof VarDecl) {
    VarDecl v = (VarDecl) stmt;
    if (symbolTable.containsKey(v.name.lexeme)) {
        errors.add("Variable '" + v.name.lexeme + "' already declared (line " + v.name.line
    }
    analyzeExpr(v.initializer);
```

```
    symbolTable.put(v.name.lexeme, "any");
}
```

Before registering a variable, the analyzer checks if it already exists in the symbol table. If it does, a duplicate declaration error is recorded. The initializer expression is analyzed first (to catch errors in it), then the variable is added to the table so subsequent statements can reference it.

SemanticAnalyzer.java — Use-Before-Declaration Check

```
} else if (expr instanceof Variable) {
    Variable v = (Variable) expr;
    if (!symbolTable.containsKey(v.name.lexeme)) {
        errors.add("Variable '" + v.name.lexeme + "' used before declaration (line " + v.nam
    }
}
```

Every time a variable is referenced in an expression, the analyzer checks the symbol table. If the variable hasn't been declared yet, an error is recorded with the line number. This catches bugs like `print(x);` before `let x = 5;`.

OpCode.java — Instruction Set

```
public enum OpCode {
    CONST_NUM, CONST_STR, CONST_CHAR, CONST_BOOL, CONST_NULL,
    LOAD, STORE,
    ADD, SUB, MUL, DIV,
    NEG, NOT,
    EQ, NEQ, GT, GTE, LT, LTE,
    AND, OR,
    JUMP, JUMP_IF_FALSE,
    PRINT,
    MAKE_ARRAY, ARRAY_LOAD, ARRAY_STORE,
    HALT
}
```

This is the complete instruction set of the CPY virtual machine. It follows a **stack-based architecture** — most instructions pop their operands from the stack and push their result back. For example, `ADD` pops two numbers and pushes their sum. `LOAD x` pushes the value of variable `x`. `STORE x` pops the top of the stack into variable `x`.

BytecodeCompiler.java — Jump Patching for If/Else

```
private void compileIf(IfStmt stmt) {
    compileExpr(stmt.condition);

    int jumpToElse = emitJump(OpCode.JUMP_IF_FALSE);  // placeholder

    compileStmt(stmt.thenBranch);

    if (stmt.elseBranch != null) {
        int jumpPastElse = emitJump(OpCode.JUMP);     // placeholder
        patchJump(jumpToElse);                         // now we know where else starts
        compileStmt(stmt.elseBranch);
        patchJump(jumpPastElse);                       // now we know where else ends
    } else {
        patchJump(jumpToElse);
    }
}

private int emitJump(OpCode jumpOp) {
    instructions.add(new Instruction(jumpOp, "0"));   // "0" is a placeholder
    return instructions.size() - 1;                   // return index for later patching
}

private void patchJump(int instructionIndex) {
    instructions.get(instructionIndex).operand = String.valueOf(instructions.size());
}
```

When compiling an `if`, the compiler doesn't yet know where the else/end is. It emits a `JUMP_IF_FALSE` with a placeholder target `"0"`, records its index, compiles the then-branch, and then **patches** the jump to point at the current instruction count. This two-pass technique (emit placeholder → patch later) is standard in single-pass bytecode compilers.

BytecodeCompiler.java — While Loop Compilation

```
private void compileWhile(WhileStmt stmt) {
    int loopStart = currentIndex();       // remember where condition starts

    compileExpr(stmt.condition);
    int jumpExit = emitJump(OpCode.JUMP_IF_FALSE);

    compileStmt(stmt.body);
    emit(OpCode.JUMP, String.valueOf(loopStart));  // jump back to condition
```

```
    patchJump(jumpExit);                    // exit jumps here (after the loop)
}
```

`loopStart` captures the instruction index before the condition is compiled. After the body, an unconditional `JUMP` sends execution back to `loopStart` to re-evaluate the condition. The `JUMP_IF_FALSE` is patched to jump past the loop when the condition becomes false.

Git Commands — Megha

```
git add Compiler/src/semantic/SemanticAnalyzer.java
git add Compiler/src/compiler/BytecodeCompiler.java
git add Compiler/src/compiler/OpCode.java
git add Compiler/src/compiler/Instruction.java
```

Part 3 — Sameer : Parser (AST Construction)

Files: `parser/Parser.java`, `ast/*.java`

The Parser takes the flat token list from the Lexer and builds an **Abstract Syntax Tree (AST)** — a tree structure that represents the grammatical structure of the program. It uses recursive descent parsing.

Parser.java — Entry Point

```
public List<Stmt> parse() {
    List<Stmt> statements = new ArrayList<>();
    while (!isAtEnd()) {
        statements.add(statement());
    }
    return statements;
}
```

The parser loops until it hits `EOF`, calling `statement()` each iteration. Each call returns one `Stmt` node (a tree node representing a statement). The result is a flat list of top-level statements, each of which may contain nested expressions and sub-statements.

Parser.java — Statement Dispatch

```
private Stmt statement() {
    if (check(TokenType.LET))    return varDeclaration();
```

```
    if (check(TokenType.IF))     return ifStatement();
    if (check(TokenType.WHILE))  return whileStatement();
    if (check(TokenType.FOR))    return forStatement();
    if (check(TokenType.PRINT))  return printStatement();
    if (check(TokenType.LBRACE)) return block();
    return assignmentOrArrayAssignment();
}
```

`check()` peeks at the current token without consuming it. Based on the leading keyword, the parser dispatches to the appropriate method. If none match, it falls through to assignment parsing. This is the classic recursive-descent pattern.

Parser.java — Variable Declaration

```
private Stmt varDeclaration() {
    consume(TokenType.LET, "Expected 'let'");
    Token name = consume(TokenType.IDENTIFIER, "Expected variable name after 'let'");
    consume(TokenType.EQUAL, "Expected '=' after variable name");
    Expr initializer = expression();
    consume(TokenType.SEMICOLON, "Expected ';' after variable declaration");
    return new VarDecl(name, initializer);
}
```

`consume()` asserts the next token is of the expected type and advances — if not, it throws a parse error with a descriptive message. This method parses `let x = <expr>;` and returns a `VarDecl` AST node holding the variable name token and the initializer expression tree.

Parser.java — If Statement with Optional Else

```
private Stmt ifStatement() {
    consume(TokenType.IF, "Expected 'if'");
    consume(TokenType.LPAREN, "Expected '(' after 'if'");
    Expr condition = expression();
    consume(TokenType.RPAREN, "Expected ')' after if condition");

    Stmt thenBranch = block();
    Stmt elseBranch = null;

    if (match(TokenType.ELSE)) {
        elseBranch = block();
    }
```

```
    return new IfStmt(condition, thenBranch, elseBranch);
}
```

`match()` consumes the token only if it matches — this makes `else` optional. If no `else` is present, `elseBranch` stays `null`. The `IfStmt` AST node stores the condition expression and both branches (either of which may be null).

Parser.java — Expression Precedence Climbing

```
private Expr expression() { return or(); }

private Expr or() {
    Expr left = and();
    while (match(TokenType.OR)) {
        Token op = previous();
        Expr right = and();
        left = new BinaryExpr(left, op, right);
    }
    return left;
}

private Expr term() {
    Expr left = factor();
    while (match(TokenType.PLUS, TokenType.MINUS)) {
        Token op = previous();
        Expr right = factor();
        left = new BinaryExpr(left, op, right);
    }
    return left;
}
```

Each method handles one precedence level and calls the next-higher-precedence method. `or` → `and` → `equality` → `comparison` → `term` → `factor` → `unary` → `primary`. The `while` loop handles left-associativity: `1 + 2 + 3` becomes `BinaryExpr(BinaryExpr(1, +, 2), +, 3)`.

Parser.java — Array Literal and Array Access

```
// In primary():
if (match(TokenType.LBRACKET)) {
    List<Expr> elements = new ArrayList<>();
    if (!check(TokenType.RBRACKET)) {
        do {
            elements.add(expression());
```

```
        } while (match(TokenType.COMMA));
    }
    consume(TokenType.RBRACKET, "Expected ']' after array literal");
    return new ArrayExpr(elements);
}

// Array access: name[expr]
if (match(TokenType.IDENTIFIER)) {
    Token name = previous();
    if (match(TokenType.LBRACKET)) {
        Expr index = expression();
        consume(TokenType.RBRACKET, "Expected ']' after array index");
        return new ArrayAccess(name, index);
    }
    return new Variable(name);
}
```

After matching an identifier, the parser peeks for `[`. If found, it parses an index expression and returns an `ArrayAccess` node. Otherwise it returns a plain `Variable` node. Array literals `[1, 2, 3]` are parsed with a `do-while` loop that keeps consuming comma-separated expressions.

Git Commands — Sameer

```
git add Compiler/src/parser/Parser.java
git add Compiler/src/ast/ArrayAccess.java
git add Compiler/src/ast/ArrayAssignment.java
git add Compiler/src/ast/ArrayExpr.java
git add Compiler/src/ast/Assignment.java
git add Compiler/src/ast/BinaryExpr.java
git add Compiler/src/ast/Block.java
git add Compiler/src/ast/Expr.java
git add Compiler/src/ast/ForStmt.java
git add Compiler/src/ast/IfStmt.java
git add Compiler/src/ast/Literal.java
git add Compiler/src/ast/PrintStmt.java
git add Compiler/src/ast/Stmt.java
git add Compiler/src/ast/UnaryExpr.java
git add Compiler/src/ast/VarDecl.java
git add Compiler/src/ast/Variable.java
git add Compiler/src/ast/WhileStmt.java
```

Part 4 — Vridhi : Lexer (Tokenization)

Files: `lexer/Lexer.java`, `lexer/Token.java`, `lexer/TokenType.java`

The Lexer is the first stage of the compiler. It reads raw source code (a plain string) and breaks it into a flat list of **tokens** — the smallest meaningful units of the language.

TokenType.java — The Token Vocabulary

```
public enum TokenType {
    LET, IF, ELSE, WHILE, FOR, PRINT,
    IDENTIFIER, NUMBER, STRING, CHAR,
    PLUS, MINUS, STAR, SLASH,
    EQUAL, EQUAL_EQUAL, BANG_EQUAL,
    GREATER, GREATER_EQUAL, LESS, LESS_EQUAL,
    AND, OR, NOT,
    LPAREN, RPAREN, LBRACE, RBRACE,
    LBRACKET, RBRACKET, SEMICOLON, COMMA,
    EOF
}
```

Every possible token in the CPY language is listed here as an enum constant. This acts as the shared vocabulary between the Lexer, Parser, and Semantic Analyzer. `EOF` is a sentinel value that signals the end of the token stream.

Token.java — The Token Data Structure

```
public class Token {
    public final TokenType type;
    public final String lexeme;
    public final int line;

    public Token(TokenType type, String lexeme, int line) {
        this.type = type;
        this.lexeme = lexeme;
        this.line = line;
    }
}
```

Each token carries three pieces of information: - `type` — what kind of token it is (e.g., `NUMBER`, `IDENTIFIER`) - `lexeme` — the exact text from the source (e.g., `"42"`, `"myVar"`) - `line` — the source line number, used for error reporting

---

Lexer.java — Keyword Map

```
private static final Map<String, TokenType> keywords = new HashMap<>();

static {
    keywords.put("let",   TokenType.LET);
    keywords.put("if",    TokenType.IF);
    keywords.put("else",  TokenType.ELSE);
    keywords.put("while", TokenType.WHILE);
    keywords.put("for",   TokenType.FOR);
    keywords.put("print", TokenType.PRINT);
    keywords.put("and",   TokenType.AND);
    keywords.put("or",    TokenType.OR);
    keywords.put("not",   TokenType.NOT);
}
```

This static map is initialized once when the class loads. When the lexer scans a word, it checks this map — if the word is a keyword it gets a specific type (e.g., `LET`), otherwise it becomes an `IDENTIFIER`. This is how `let x = 5` knows `let` is a keyword and `x` is a variable name.

Lexer.java — Main Scan Loop

```
public List<Token> scanTokens() {
    while (!isAtEnd()) {
        start = current;
        scanToken();
    }
    tokens.add(new Token(TokenType.EOF, "", line));
    return tokens;
}
```

`start` marks the beginning of the current token being scanned. `current` advances as characters are consumed. Each iteration of the loop resets `start` to `current`, then calls `scanToken()` to classify the next character. After the source is exhausted, an `EOF` token is appended so the parser always has a clean termination signal.

Lexer.java — Two-Character Operators

```
case '=':
    addToken(match('=') ? TokenType.EQUAL_EQUAL : TokenType.EQUAL);
    break;
case '!':
    addToken(match('=') ? TokenType.BANG_EQUAL : TokenType.NOT);
```

```
    break;
case '>':
    addToken(match('=') ? TokenType.GREATER_EQUAL : TokenType.GREATER);
    break;
```

`match(char)` peeks at the next character and consumes it only if it matches. This is how `=` vs `==`, `!` vs `!=`, and `>` vs `>=` are disambiguated in a single pass without backtracking.

Lexer.java — Number Scanning

```
private void number() {
    while (!isAtEnd() && isDigit(peek())) advance();

    if (!isAtEnd() && peek() == '.' && isDigit(peekNext())) {
        advance(); // consume '.'
        while (!isAtEnd() && isDigit(peek())) advance();
    }

    addToken(TokenType.NUMBER);
}
```

Scans integer digits first. Then checks if the next character is `.` followed by another digit — if so, it's a decimal number. `peekNext()` looks two characters ahead to avoid treating `5.` (trailing dot) as a float. The entire matched text (e.g., `"3.14"`) is stored as the lexeme.

Lexer.java — Comment Handling

```
case '/':
    if (peek() == '/') {
        while (!isAtEnd() && peek() != '\n') advance();
    } else {
        addToken(TokenType.SLASH);
    }
    break;
```

When `/` is seen, the lexer peeks at the next character. If it's another `/`, the rest of the line is a comment and is silently consumed without producing any token. Otherwise, a `SLASH` (division) token is emitted.

Git Commands — Vridhi

```
git add Compiler/src/lexer/Lexer.java
git add Compiler/src/lexer/Token.java
git add Compiler/src/lexer/TokenType.java
```

Pipeline Summary

```
source.cpy


[Lexer]  →  List<Token>


[Parser]  →  List<Stmt>  (AST)


[SemanticAnalyzer]  →  validates AST


[BytecodeCompiler]  →  List<Instruction>


[BytecodeWriter]  →  program.cpyc


[BytecodeReader]  →  List<Instruction>


[VM]  →  program output
```